🎯 GLA UNIVERSITY — DAY 1 POST-ASSESSMENT 50 MCQS (OFFICIAL INSTRUCTOR MASTER)

High Performance Computing & Heterogeneous AI Systems (`GLA-HPC-BCSG701`)

Lead Trainer: Shivam Gupta | **Target Session:** Day 1 Conclusion (Evaluative Mastery Assessment)
Total Questions: 50 Multi-Choice Questions | **Target Pass Rate:** 85% – 95% Learning Gain
Evaluation Specifications: Official 7-Column University Examination Spec

---

📌 Pedagogical Architecture & Delta Evaluation Strategy

This 50-question post-assessment is administered at the conclusion of Day 1 (04:00 PM – 05:00 PM). While the morning Pre-Assessment measured incoming baseline competence (target 40%–60%), this Post-Assessment evaluates mastery of the exact microarchitectural, system administration, cluster networking, and oneAPI concepts delivered across the 6-hour intensive instruction.

Module Distribution Breakdown (50 Questions Total):

- **Module 1: HPC Principles & Architectures:** Questions 01 – 08 (8 Questions: FLOPS derivation, Amdahl/Gustafson laws, Roofline model, topologies, RDMA)
- **Module 2: Intel HPC Architecture & Hardware Accelerators:** Questions 09 – 15 (7 Questions: AMX tile registers, AVX-512 predication, SNC clustering, thermal licenses, Gaudi)
- **Module 3: Linux Operating System Foundations for HPC:** Questions 16 – 22 (7 Questions: VFS inodes, octal umask derivation, SGID bits, zombies vs orphans, page cache)
- **Module 4: Process Management, Signals & Resource Limits:** Questions 23 – 29 (7 Questions: POSIX signals, OOM score tuning, CPU affinity taskset, ulimits, load average)
- **Module 5: Secure Remote Access & Cluster Networking:** Questions 30 – 36 (7 Questions: Asymmetric SSH keypairs, strict permissions, rsync delta, agent forwarding, Jumbo Frames)
- **Module 6: Advanced Linux Bash Shell Scripting for HPC Automation:** Questions 37 – 43 (7 Questions: Defensive set -euo pipefail, traps, parameter expansion, awk, redirection)
- **Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming:** Questions 44 – 50 (7 Questions: oneAPI vision, Base vs HPC toolkits, setvars.sh, SYCL single-source, USM malloc)

Question 01 [Module 1: HPC Principles & Architectures | Medium | Numerical & Hardware Derivation]

Topic: Theoretical Peak Floating-Point Performance (FLOPS)
Question: An HPC dual-socket compute node contains two Intel Xeon Platinum 8480+ processors. Each processor has 56 cores running at a base vector frequency of 2.0 GHz. Each core has two 512-bit FMA (Fused Multiply-Add) execution units for double precision (FP64). What is the theoretical peak double-precision performance of this dual-socket node?

- **a)** 3.58 TFLOPS
- **b)** 7.168 TFLOPS

- **c)** 14.336 TFLOPS
- **d)** 28.672 TFLOPS

> **TIP**
> **Correct Answer: [b]** — 7.168 TFLOPS
> **Technical Explanation:** Each 512-bit AVX-512 vector register holds 8 FP64 doubles (512 / 64 = 8). An FMA operation counts as 2 FLOPS (one multiply + one add). With 2 FMA units per core, each core performs: 2 FMAs * 8 doubles * 2 FLOPS = 32 FP64 FLOPS per clock cycle. For 56 cores * 2 sockets = 112 cores at 2.0 GHz: Peak = 112 * 2.0 * 10^9 cycles/sec * 32 FLOPS/cycle = 7,168 GFLOPS = 7.168 TFLOPS.

Question 02 [Module 1: HPC Principles & Architectures | Medium | Mathematical Derivation]

Topic: Amdahl's Law Speedup Calculation
Question: A molecular dynamics simulation code spends 15% of its total execution time in sequential file I/O and initialization that cannot be parallelized. What is the maximum theoretical speedup of this application when running on a 64-core cluster node according to Amdahl's Law?

- **a)** 6.12x
- **b)** 10.0x
- **c)** 5.77x
- **d)** 64.0x

> **TIP**
> **Correct Answer: [a]** — 6.12x
> **Technical Explanation:** By Amdahl's Law: Speedup = 1 / [S + (P / N)], where S = 0.15 (sequential fraction), P = 0.85 (parallel fraction), and N = 64 (cores). Speedup = 1 / [0.15 + (0.85 / 64)] = 1 / [0.15 + 0.01328] = 1 / 0.16328 ≈ 6.124x.

Question 03 [Module 1: HPC Principles & Architectures | Hard | Theoretical Architecture]

Topic: Gustafson's Law vs Amdahl's Law
Question: Why does Gustafson's Law offer a more realistic speedup model than Amdahl's Law for modern large-scale supercomputing clusters running scientific simulations?

- **a)** Gustafson's Law assumes the number of processor cores remains constant while clock speed increases
- **b)** Gustafson's Law models weak scaling, where the problem size increases proportionally with the number of compute processors
- **c)** Gustafson's Law proves that sequential fractions can be executed in zero time on quantum processors
- **d)** Gustafson's Law only applies to GPUs, whereas Amdahl's Law only applies to CPUs

> **TIP**
> **Correct Answer: [b]** — Gustafson's Law models weak scaling, where the problem size increases proportionally with the number of compute processors
> **Technical Explanation:** Amdahl's Law models 'strong scaling' (fixed problem size distributed across more processors, where serial overhead dominates). Gustafson's Law models 'weak scaling' (scaled speedup), recognizing that as computing power scales up, scientists expand the problem size (higher grid resolution, more particles), making the parallel fraction grow while sequential overhead stays fixed.

Question 04 [Module 1: HPC Principles & Architectures | Hard | Performance Analysis]

Topic: Roofline Model & Operational Intensity
Question: Under the Roofline Performance Model, an algorithm has an arithmetic intensity of 0.25 FLOPs/byte. The target HPC compute node has a peak memory bandwidth of 400 GB/s and a peak compute performance of 8.0 TFLOPS. What is the maximum performance attainable by this algorithm, and what is its limiting bound?

- **a)** 100 GFLOPS, Memory Bandwidth Bound
- **b)** 2.0 TFLOPS, Compute Bound
- **c)** 8.0 TFLOPS, Compute Bound
- **d)** 1.6 TFLOPS, Latency Bound

> **TIP**
> **Correct Answer: [a]** — 100 GFLOPS, Memory Bandwidth Bound
> **Technical Explanation:** Under the Roofline Model: Attainable Performance = min(Peak Compute, Arithmetic Intensity * Peak Memory Bandwidth). Here: 0.25 FLOPs/byte * 400 GB/s = 100 GFLOPS. Since 100 GFLOPS (0.1 TFLOPS) is far below the peak compute ceiling of 8.0 TFLOPS, the application is strictly Memory Bandwidth Bound.

Question 05 [Module 1: HPC Principles & Architectures | Medium | Cluster Networking]

Topic: Interconnect Network Topologies & Bisection Bandwidth
Question: What is the defining characteristic of a 'Full Bisection Bandwidth' (non-blocking) Fat-Tree interconnect in an enterprise HPC cluster?

- **a)** All nodes in the cluster share a single physical copper ethernet cable
- **b)** Any arbitrary half of the compute nodes can communicate simultaneously with the other half at full network line rate without oversubscription
- **c)** The network switches turn off automatically when compute nodes are idle
- **d)** Communication latency increases by a factor of 10 for every additional rack added

> **TIP**
> **Correct Answer: [b]** — Any arbitrary half of the compute nodes can communicate simultaneously with the other half at full network line rate without oversubscription
> **Technical Explanation:** A non-blocking Fat-Tree topology with full bisection bandwidth ensures that the aggregate bandwidth between any partition of half the cluster and the remaining half equals the sum of line rates of all nodes in that half (1:1 oversubscription ratio), eliminating interconnect congestion bottlenecks.

Question 06 [Module 1: HPC Principles & Architectures | Medium | Cluster Networking]

Topic: InfiniBand vs Traditional Ethernet in HPC
Question: Why is InfiniBand with RDMA (Remote Direct Memory Access) preferred over standard TCP/IP Ethernet for distributed HPC applications like MPI?

- **a)** InfiniBand uses less electricity because it operates without network switches
- **b)** RDMA allows network adapters to transfer data directly between application memories across nodes without CPU OS kernel context switches and copying
- **c)** TCP/IP cannot transmit floating-point data across physical networks
- **d)** InfiniBand cables only support wireless radio transmission

> **TIP**
>
> **Correct Answer: [b]** — RDMA allows network adapters to transfer data directly between application memories across nodes without CPU OS kernel context switches and copying
>
> **Technical Explanation:** RDMA bypasses the operating system kernel entirely. The Host Channel Adapter (HCA) transfers data directly from user-space virtual memory on one node to user-space memory on another node (zero-copy), achieving sub-microsecond latency and zero host CPU overhead compared to standard TCP/IP socket buffer copying.

Question 07 [Module 1: HPC Principles & Architectures | Easy | System Architecture]

Topic: Heterogeneous Compute Node Architecture
Question: In a modern heterogeneous HPC supercomputing node (CPU + GPU/Accelerator), what is the primary division of architectural responsibility?

- **a)** The GPU handles file I/O and OS interrupts, while the CPU performs deep learning matrix multiplication
- **b)** The CPU acts as the latency-optimized orchestrator/control plane, while the GPU acts as the high-throughput parallel data execution engine
- **c)** The CPU and GPU must run completely identical instruction sets at the exact same clock speed
- **d)** The GPU replaces the system motherboard and BIOS entirely

> **TIP**
>
> **Correct Answer: [b]** — The CPU acts as the latency-optimized orchestrator/control plane, while the GPU acts as the high-throughput parallel data execution engine
>
> **Technical Explanation:** CPUs feature large caches, sophisticated branch predictors, and high single-thread clock frequencies, making them ideal for complex control logic, OS management, and I/O orchestration. GPUs feature thousands of SIMD/SIMT cores designed for massive data-parallel, high-throughput compute kernels.

Question 08 [Module 1: HPC Principles & Architectures | Hard | HPC Storage Architecture]

Topic: Parallel File Systems (Lustre / GPFS / BeeGFS)
Question: How does a distributed parallel file system (such as Lustre or BeeGFS) prevent I/O bottlenecks when thousands of cluster compute cores write checkpoints simultaneously?

- **a)** By compressing all files into a single ZIP archive on the head node
- **b)** By separating metadata operations (MDT) from data storage (OST) and striping file chunks across multiple physical storage servers concurrently
- **c)** By disabling file write permissions and running purely in RAM
- **d)** By requiring each compute node to write to an individual USB flash drive

> **TIP**
>
> **Correct Answer: [b]** — By separating metadata operations (MDT) from data storage (OST) and striping file chunks across multiple physical storage servers concurrently
>
> **Technical Explanation:** Parallel file systems decouple metadata servers (MDS/MDT, managing directory paths and permissions) from object storage servers/targets (OSS/OST). Large files are stripped across dozens of OSTs, allowing thousands of nodes to stream data in parallel without serializing at a single storage head.

Question 09 [Module 2: Intel HPC Architecture & Hardware Accelerators | Hard | Microarchitecture Rigor]

Topic: Intel Advanced Matrix Extensions (Intel AMX)
Question: In Intel 4th and 5th Gen Xeon Scalable processors (Sapphire Rapids / Emerald Rapids), what are the two core hardware components of Intel AMX?

- **a)** L1 Data Cache and Translation Lookaside Buffer (TLB)
- **b)** Eight 1KB 2D Tile Registers (TMM0-TMM7) and the Tile Matrix Multiply Unit (TMUL)
- **c)** PCIe Gen 5 bus controller and CXL memory expander
- **d)** AVX-256 integer ALU and IEEE-754 floating-point divider

> **TIP**
> **Correct Answer: [b]** — Eight 1KB 2D Tile Registers (TMM0-TMM7) and the Tile Matrix Multiply Unit (TMUL)
> **Technical Explanation:** Intel AMX introduces two physical components: (1) Tiles: eight 2D register files (TMM0 to TMM7), each holding 1KB of matrix data (8KB total), and (2) TMUL (Tile Matrix Multiply Unit): an execution accelerator that computes dot products on BF16 or INT8 tiles in a single instruction cycle.

Question 10 [Module 2: Intel HPC Architecture & Hardware Accelerators | Medium | SIMD Vectorization]

Topic: AVX-512 Masking & Vector Predication
Question: How do AVX-512 opmask registers (k0 through k7) improve performance when vectorizing conditional loops (`if-else` branches) in HPC scientific code?

- **a)** They automatically increase the CPU core clock frequency during `if` statements
- **b)** They allow conditional vector operations via bitmask predication, executing active vector lanes while suppressing inactive lanes without branch misprediction penalties
- **c)** They convert 32-bit floats into 64-bit doubles automatically
- **d)** They disable cache coherency across cores during conditional checks

> **TIP**
> **Correct Answer: [b]** — They allow conditional vector operations via bitmask predication, executing active vector lanes while suppressing inactive lanes without branch misprediction penalties
> **Technical Explanation:** Traditional SIMD struggled with conditional branches because branches caused serialization. AVX-512 introduces 8 dedicated opmask registers (k0-k7). A vector comparison produces a bitmask; subsequent vector instructions use this mask to update only true lanes while preserving or zeroing false lanes without branching.

Question 11 [Module 2: Intel HPC Architecture & Hardware Accelerators | Hard | NUMA & Memory Hierarchy]

Topic: Sub-NUMA Clustering (SNC) & UPI Interconnect
Question: What is the primary architectural benefit of configuring Sub-NUMA Clustering (SNC-2 or SNC-4) in the BIOS of a dual-socket Intel Xeon server for HPC workloads?

- **a)** It disables DDR5 ECC checking to achieve higher memory clock rates
- **b)** It partitions the physical processor and its memory controllers into smaller localized NUMA domains, reducing average L3 cache and memory access latency
- **c)** It forces all MPI ranks to communicate using Ethernet instead of shared memory

- **d)** It allows Windows and Linux to boot simultaneously on separate cores

> **TIP**
> **Correct Answer: [b]** — It partitions the physical processor and its memory controllers into smaller localized NUMA domains, reducing average L3 cache and memory access latency
> **Technical Explanation:** SNC divides the physical cores and memory controllers of a single Xeon socket into localized clusters (e.g., SNC-2 splits 1 socket into 2 NUMA domains). Threads accessing local memory within their SNC cluster experience shorter ring/mesh traversal hops and significantly lower memory access latency.

Question 12 [Module 2: Intel HPC Architecture & Hardware Accelerators | Hard | Silicon & Power Management]

Topic: AVX Frequency Scaling & Power Licensing
Question: When an HPC node executes heavy AVX-512 512-bit FMA workloads, why might the Linux `turbostat` utility report a core frequency slightly lower than the scalar base frequency?

- **a)** The Linux kernel has encountered a hardware fault in the CPU voltage regulator
- **b)** Intel Xeon processors switch to an optimized AVX-512 power license level with dynamic voltage/frequency scaling to prevent exceeding thermal and electrical design current (TDP/ICCMax) limits
- **c)** AVX-512 instructions disable hyperthreading permanently
- **d)** The system RAM runs out of bandwidth and turns off CPU cores

> **TIP**
> **Correct Answer: [b]** — Intel Xeon processors switch to an optimized AVX-512 power license level with dynamic voltage/frequency scaling to prevent exceeding thermal and electrical design current (TDP/ICCMax) limits
> **Technical Explanation:** AVX-512 instructions draw significantly higher electrical current per cycle through the wide 512-bit vector units. Intel processors implement dynamic voltage-frequency license levels (Scalar, AVX2, AVX-512 light, AVX-512 heavy). When heavy vector instructions are detected, frequency is adjusted dynamically to keep the core within electrical and thermal envelopes.

Question 13 [Module 2: Intel HPC Architecture & Hardware Accelerators | Medium | Hardware Accelerators]

Topic: Intel Data Center GPU Max (Ponte Vecchio)
Question: Which advanced packaging technology and memory architecture are utilized in the Intel Data Center GPU Max Series (formerly code-named Ponte Vecchio) to deliver massive HPC throughput?

- **a)** Single monolithic silicon die with GDDR6 memory
- **b)** EMIB (Embedded Multi-die Interconnect Bridge) and Foveros 3D die stacking with High Bandwidth Memory (HBM2e)
- **c)** Wire-bonded packaging with standard DDR4 SODIMM sockets
- **d)** Optane Persistent Memory connected via SATA interface

> **TIP**
>
> **Correct Answer: [b]** — EMIB (Embedded Multi-die Interconnect Bridge) and Foveros 3D die stacking with High Bandwidth Memory (HBM2e)
>
> **Technical Explanation:** Intel GPU Max (Ponte Vecchio) integrates over 100 billion transistors across 47 active tiles using EMIB 2.5D horizontal bridging and Foveros 3D vertical stacking, integrated directly with up to 128 GB of ultra-wide HBM2e memory delivering over 3.2 TB/s aggregate bandwidth.

Question 14 [Module 2: Intel HPC Architecture & Hardware Accelerators | Hard | AI Accelerator Architecture]

Topic: Intel Gaudi AI Accelerators & Interconnect
Question: What unique networking architectural design distinguishes Intel Gaudi 2 / Gaudi 3 AI processors from traditional GPU clusters?

- **a)** They do not require power cables and run on solar energy
- **b)** They integrate multiple 100GbE / 200GbE RoCEv2 (RDMA over Converged Ethernet) ports directly on the silicon accelerator package, eliminating external PCIe NICs
- **c)** They only communicate over USB 3.0 serial connections
- **d)** They prohibit multi-node distributed training across racks

> **TIP**
>
> **Correct Answer: [b]** — They integrate multiple 100GbE / 200GbE RoCEv2 (RDMA over Converged Ethernet) ports directly on the silicon accelerator package, eliminating external PCIe NICs
>
> **Technical Explanation:** Intel Gaudi processors feature on-die native RoCEv2 Ethernet integration (e.g., 24 ports of 100GbE on Gaudi 2; 24 ports of 200GbE on Gaudi 3). This enables all-to-all scale-up within a node and scale-out across cluster nodes using standard Ethernet switches without needing expensive proprietary external PCIe network adapters.

Question 15 [Module 2: Intel HPC Architecture & Hardware Accelerators | Easy | Applied Command Line]

Topic: Hardware Feature Discovery via Linux CLI
Question: You run `lscpu | grep -i flags` on an HPC cluster node and see the flags `avx512f`, `amx_tile`, and `amx_bf16`. What does this indicate about the host CPU?

- **a)** It is an older AMD Opteron processor from 2012
- **b)** It is an Intel Xeon Scalable processor (Sapphire Rapids or newer) with hardware support for AVX-512 and matrix multiplication acceleration for BF16 precision
- **c)** It is an ARM Cortex-A53 processor without vector extensions
- **d)** The system is running on a software emulation layer without hardware acceleration

> **TIP**
>
> **Correct Answer: [b]** — It is an Intel Xeon Scalable processor (Sapphire Rapids or newer) with hardware support for AVX-512 and matrix multiplication acceleration for BF16 precision
>
> **Technical Explanation:** The `avx512f` flag represents AVX-512 Foundation instructions, while `amx_tile` and `amx_bf16` confirm the presence of Intel Advanced Matrix Extensions (AMX) introduced in 4th Gen Intel Xeon Scalable processors (Sapphire Rapids) for accelerating Bfloat16 AI training and inference.

Question 16 [Module 3: Linux Operating System Foundations for HPC | Medium | Kernel & OS Internals]

Topic: Linux Virtual File System (VFS) & Inodes
Question: In the Linux filesystem architecture, what happens at the metadata level when an engineer creates a hard link to an existing file using `ln fileA linkB`?

- **a)** A new inode is created with a duplicate copy of the data blocks
- **b)** A new directory entry `linkB` is created pointing to the identical inode number as `fileA`, and the inode's link count is incremented by 1
- **c)** The original file `fileA` is deleted immediately
- **d)** The file is converted into a Windows shortcut file

> **TIP**
> **Correct Answer: [b]** — A new directory entry `linkB` is created pointing to the identical inode number as `fileA`, and the inode's link count is incremented by 1
> **Technical Explanation:** A hard link creates an additional directory entry (dentry) pointing to the exact same physical inode on disk. No data blocks are duplicated. The inode's hard link reference counter is incremented by 1. Deleting either file merely decrements the counter; data is only freed when the link count reaches 0 and no processes hold open file descriptors.

Question 17 [Module 3: Linux Operating System Foundations for HPC | Medium | Applied Systems Administration]

Topic: File Permissions & Octal Umask Derivation
Question: An HPC administrator sets `umask 027` in `/etc/profile`. What will be the exact default permission strings and octal modes for newly created directories and regular files respectively?

- **a)** Directories: `rwxr-x---` (750); Regular Files: `rw-r-----` (640)
- **b)** Directories: `rwxrwxrwx` (777); Regular Files: `rw-rw-rw-` (666)
- **c)** Directories: `rwxr-xr-x` (755); Regular Files: `rw-r--r--` (644)
- **d)** Directories: `rwx------` (700); Regular Files: `rw-------` (600)

> **TIP**
> **Correct Answer: [a]** — Directories: `rwxr-x---` (750); Regular Files: `rw-r-----` (640)
> **Technical Explanation:** Base permission for directories is 777 (rwxrwxrwx). With umask 027: 777 - 027 = 750 (`rwxr-x---`). Base permission for files is 666 (rw-rw-rw-). With umask 027: 666 - 026 (masked bits) = 640 (`rw-r-----`). Files never receive execute bits by default upon creation.

Question 18 [Module 3: Linux Operating System Foundations for HPC | Hard | File Permissions]

Topic: Special Permission Bits (SGID on Directories)
Question: Why does an HPC system administrator apply the SGID bit to a collaborative project directory via `chmod 2775 /shared/research_data`?

- **a)** To allow any student to delete other students' files without permission
- **b)** To ensure all new files and subdirectories created inside `/shared/research_data` automatically inherit the group ownership of the parent directory rather than the creator's primary group

- **c)** To encrypt the directory with AES-256 automatically
- **d)** To force the directory to execute as root

> **TIP**
> **Correct Answer: [b]** — To ensure all new files and subdirectories created inside `/shared/research_data` automatically inherit the group ownership of the parent directory rather than the creator's primary group
> **Technical Explanation:** When the SGID (Set Group ID, octal bit 2) is set on a directory, any file or subdirectory created within it automatically inherits the group ID of the directory, rather than the primary group ID of the user creating the file. Combined with `775`, this ensures seamless collaboration among group members.

Question 19 [Module 3: Linux Operating System Foundations for HPC | Medium | Process Architecture]

Topic: Process Lifecycle: Zombie vs Orphan Processes
Question: In an HPC simulation pipeline, what is the precise technical difference between a 'Zombie' process and an 'Orphan' process in Linux?

- **a)** A Zombie process consumes 100% CPU; an Orphan process consumes 100% RAM
- **b)** A Zombie process has completed execution but its parent has not yet read its exit status via `waitpid()`; an Orphan process is still running after its parent has terminated and is adopted by PID 1 (systemd)
- **c)** Zombie processes only exist on Windows; Orphan processes only exist on Linux
- **d)** An Orphan process has been killed by the OOM killer; a Zombie process is waiting for network packets

> **TIP**
> **Correct Answer: [b]** — A Zombie process has completed execution but its parent has not yet read its exit status via `waitpid()`; an Orphan process is still running after its parent has terminated and is adopted by PID 1 (systemd)
> **Technical Explanation:** A Zombie process (state 'Z' in `ps`) has finished execution and released its memory and file descriptors, but its process table entry remains so the parent can inspect its termination status code. An Orphan process is an actively running process whose original parent died; the Linux kernel immediately re-parents it to PID 1 (`init` or `systemd`).

Question 20 [Module 3: Linux Operating System Foundations for HPC | Easy | Kernel Observability]

Topic: The /proc Pseudo-Filesystem
Question: Where in the Linux virtual filesystem can an HPC researcher programmatically inspect real-time kernel memory statistics, active CPU topology, and hardware interrupts without invoking binary tools?

- **a)** Under the `/etc/sysconfig` folder
- **b)** Under the `/proc` virtual pseudo-filesystem (e.g., `/proc/meminfo`, `/proc/cpuinfo`, `/proc/interrupts`)
- **c)** Inside the `/tmp` temporary swap partition
- **d)** Within the `/usr/share/man` documentation directories

> **TIP**
> **Correct Answer: [b]** — Under the `/proc` virtual pseudo-filesystem (e.g., `/proc/meminfo`, `/proc/cpuinfo`, `/proc/interrupts`)
> **Technical Explanation:** The `/proc` filesystem is a virtual pseudo-filesystem created dynamically in RAM by the Linux kernel. It acts as an interactive window into live kernel data structures, exporting hardware configurations, CPU topology, memory state, and per-process telemetry (`/proc/[pid]/`).

Question 21 [Module 3: Linux Operating System Foundations for HPC | Medium | Memory Management]

Topic: Linux Page Cache & Memory Accounting
Question: When running `free -h` on an HPC server, the `used` memory is 12 GB, but `buff/cache` is 500 GB out of 512 GB total RAM. Why should the administrator NOT panic?

- **a)** The RAM is defective and must be physically replaced
- **b)** Linux aggressively uses free RAM as Page Cache to accelerate disk I/O; this cached memory is immediately reclaimable if compute applications request memory
- **c)** A memory leak virus has infected the Linux kernel
- **d)** The system has swapped all data to the local hard drive

> **TIP**
> **Correct Answer: [b]** — Linux aggressively uses free RAM as Page Cache to accelerate disk I/O; this cached memory is immediately reclaimable if compute applications request memory
> **Technical Explanation:** Linux follows the design philosophy that 'unused RAM is wasted RAM'. It caches recently read and written disk blocks in memory. The `available` column indicates memory that can be allocated immediately to applications because the kernel will discard clean page cache blocks instantly without pausing execution.

Question 22 [Module 3: Linux Operating System Foundations for HPC | Hard | Software Environment]

Topic: Dynamic Linker & Shared Object Resolution
Question: An engineer compiles an Intel oneAPI application, but running `./vector_add` fails with: `error while loading shared libraries: libmkl_intel_lp64.so.2: cannot open shared object file`. What is the root cause and remedy?

- **a)** The file system is corrupted and must be reformatted
- **b)** The dynamic linker (`ld.so`) cannot locate oneAPI runtime libraries; the remedy is to set `LD_LIBRARY_PATH` by running `source /opt/intel/oneapi/setvars.sh`
- **c)** The CPU does not support 64-bit computation
- **d)** The binary was compiled for ARM architecture instead of x86_64

> **TIP**
> **Correct Answer: [b]** — The dynamic linker (`ld.so`) cannot locate oneAPI runtime libraries; the remedy is to set `LD_LIBRARY_PATH` by running `source /opt/intel/oneapi/setvars.sh`
> **Technical Explanation:** Dynamic binaries inspect `LD_LIBRARY_PATH` and cache `/etc/ld.so.cache` at runtime to resolve `.so` shared objects. Intel oneAPI libraries reside in non-standard paths like `/opt/intel/oneapi/mkl/latest/lib/intel64`. Sourcing `setvars.sh` updates `LD_LIBRARY_PATH` so the Linux dynamic linker can locate the required libraries.

Question 23 [Module 4: Process Management, Signals & Resource Limits | Easy | Signal Handling]

Topic: POSIX Signals: SIGTERM vs SIGKILL
Question: Why should an HPC cluster administrator always attempt to terminate an errant job with `kill -15 <pid>` (SIGTERM) before resorting to `kill -9 <pid>` (SIGKILL)?

- **a)** SIGTERM executes 10 times faster than SIGKILL
- **b)** SIGTERM can be caught by the application's signal handler to flush buffers, save scientific checkpoints, and close open network sockets cleanly; SIGKILL immediately destroys the process table entry without cleanup
- **c)** SIGKILL requires root permissions while SIGTERM does not
- **d)** SIGTERM only pauses the process for 5 seconds

> **TIP**
> **Correct Answer: [b]** — SIGTERM can be caught by the application's signal handler to flush buffers, save scientific checkpoints, and close open network sockets cleanly; SIGKILL immediately destroys the process table entry without cleanup
> **Technical Explanation:** SIGTERM (signal 15) is the polite termination request. Well-designed HPC codes catch SIGTERM to write recovery checkpoint files and release cluster locks. SIGKILL (signal 9) cannot be caught, blocked, or handled; the kernel terminates the process instantly, potentially leaving temporary files corrupted or cluster resources locked.

Question 24 [Module 4: Process Management, Signals & Resource Limits | Medium | Process Control]

Topic: Process Suspension: SIGSTOP and SIGCONT
Question: An urgent high-priority interactive debug job arrives on a shared node. How can an administrator temporarily suspend a running background simulation without terminating it or losing its in-memory calculation state, and resume it later?

- **a)** `kill -STOP <pid>` to pause execution; `kill -CONT <pid>` to resume execution
- **b)** `kill -9 <pid>` to pause; `kill -1 <pid>` to resume
- **c)** `nice -n 20 <pid>` to pause; `renice -n 0 <pid>` to resume
- **d)** `chmod 000 <binary>` to pause; `chmod 755 <binary>` to resume

> **TIP**
> **Correct Answer: [a]** — `kill -STOP <pid>` to pause execution; `kill -CONT <pid>` to resume execution
> **Technical Explanation:** SIGSTOP (signal 19) halts the execution of a process immediately. The process leaves the CPU runqueue and transitions to the Stopped ('T') state while its full memory state remains intact in RAM. When ready to resume, sending SIGCONT (signal 18) returns it to the runnable state.

Question 25 [Module 4: Process Management, Signals & Resource Limits | Hard | Kernel Memory Management]

Topic: Linux Out-of-Memory (OOM) Killer Architecture
Question: When a node exhausts physical RAM and swap, how does the Linux OOM Killer decide which process to kill, and how can a critical cluster manager daemon (`slurmd`) be protected?

- **a)** It kills processes randomly; daemons cannot be protected
- **b)** It calculates a heuristic score based on RAM usage and runtime (`/proc/[pid]/oom_score`); daemons can be protected by writing `-1000` to `/proc/[pid]/oom_score_adj`

- **c)** It always kills the process with PID 1 first
- **d)** It deletes disk files until memory becomes free

> **TIP**
> **Correct Answer: [b]** — It calculates a heuristic score based on RAM usage and runtime (`/proc/[pid]/oom_score`); daemons can be protected by writing `-1000` to `/proc/[pid]/oom_score_adj`
> **Technical Explanation:** The Linux kernel computes an `oom_score` (0-1000) for every process based on the proportion of RAM consumed. The process with the highest score is targeted. Setting `/proc/[pid]/oom_score_adj` to `-1000` grants complete immunity, guaranteeing the kernel will never terminate that process during memory exhaustion.

Question 26 [Module 4: Process Management, Signals & Resource Limits | Hard | Performance Tuning]

Topic: CPU Affinity & NUMA Binding (`taskset` & `numactl`)
Question: Why is binding an HPC simulation thread to specific physical CPU cores using `taskset -c 0-7` or `numactl --physcpubind=0-7 --membind=0` critical for achieving reproducible benchmarks?

- **a)** It prevents the operating system from updating the system clock
- **b)** It eliminates cache thrashing and inter-socket NUMA latency caused by the OS scheduler randomly migrating threads between different physical sockets
- **c)** It doubles the PCIe bus bandwidth automatically
- **d)** It bypasses CPU thermal sensors to prevent throttling

> **TIP**
> **Correct Answer: [b]** — It eliminates cache thrashing and inter-socket NUMA latency caused by the OS scheduler randomly migrating threads between different physical sockets
> **Technical Explanation:** When the Linux Completely Fair Scheduler (CFS) migrates a thread from Core 0 on Socket 0 to Core 32 on Socket 1, the thread loses all warm L1/L2 cache data. Additionally, memory allocations located on Socket 0 must now traverse the UPI interconnect, quadrupling memory latency. Thread and memory pinning prevents this penalty.

Question 27 [Module 4: Process Management, Signals & Resource Limits | Medium | Process Scheduling]

Topic: Process Niceness & CFS Scheduling Priority
Question: A student launches a CPU-intensive data pre-processing script on a shared login headnode using `nice -n 19 python preprocess.py`. What does this niceness value dictate to the Linux scheduler?

- **a)** It assigns the highest possible real-time priority, preempting all other users
- **b)** It sets the lowest possible priority, causing the process to yield CPU cycles whenever any normal or higher-priority job requires computing time
- **c)** It limits the script to run only on CPU 19
- **d)** It allocates 19 GB of dedicated RAM to the Python process

> **TIP**
>
> **Correct Answer: [b]** — It sets the lowest possible priority, causing the process to yield CPU cycles whenever any normal or higher-priority job requires computing time
>
> **Technical Explanation:** Linux nice values range from -20 (highest priority, least nice) to +19 (lowest priority, most nice). A niceness of +19 instructs the Completely Fair Scheduler (CFS) to grant the process the lowest proportional CPU time share, running only when the system has idle cycles.

Question 28 [Module 4: Process Management, Signals & Resource Limits | Medium | Resource Configuration]

Topic: System Resource Limits (`ulimit`) for HPC
Question: Why do HPC administrators explicitly set `ulimit -s unlimited` and `ulimit -l unlimited` in cluster batch environments before running OpenMP or MPI jobs?

- **a)** To allow unlimited network internet downloads
- **b)** To prevent segmentation faults caused by large thread-private stack allocations and enable InfiniBand drivers to lock physical memory pages (pinning) for RDMA transfers
- **c)** To allow users to create an infinite number of user accounts
- **d)** To permanently disable the Linux firewall

> **TIP**
>
> **Correct Answer: [b]** — To prevent segmentation faults caused by large thread-private stack allocations and enable InfiniBand drivers to lock physical memory pages (pinning) for RDMA transfers
>
> **Technical Explanation:** Default Linux stack limits (typically 8MB) cause instant stack overflow segmentation faults in OpenMP threads allocating large local scientific arrays. Similarly, InfiniBand RDMA requires locking physical memory (`mlock`) to prevent the kernel from paging out buffers; an unlimited locked-memory limit is mandatory.

Question 29 [Module 4: Process Management, Signals & Resource Limits | Medium | Cluster Monitoring]

Topic: Linux Load Average Telemetry Interpretation
Question: An HPC compute node with 64 physical CPU cores reports a 1-minute load average of 80.0. What does this metric signify regarding system load?

- **a)** 80% of the RAM has been consumed by the operating system
- **b)** All 64 cores are fully occupied, and an average of 16 process threads are queued waiting for CPU or stuck in uninterruptible disk I/O (D state)
- **c)** The CPU temperature has reached 80 degrees Celsius
- **d)** The system network interface is transmitting at 80 Gbps

> **TIP**
>
> **Correct Answer: [b]** — All 64 cores are fully occupied, and an average of 16 process threads are queued waiting for CPU or stuck in uninterruptible disk I/O (D state)
>
> **Technical Explanation:** Load average measures the average number of processes in the runnable (R) state plus processes in the uninterruptible sleep (D) state. On a 64-core system, a load of 64.0 represents 100% full capacity. A load of 80.0 means 64 tasks are executing and 16 tasks are waiting in queue, indicating CPU or I/O oversubscription.

Question 30 [Module 5: Secure Remote Access & Cluster Networking | Easy | Security & Cryptography]

Topic: Asymmetric Cryptography & SSH Key Pairs
Question: When configuring SSH key-based authentication for an HPC cluster, where must the user's Public Key and Private Key reside respectively?

- **a)** Both the public and private keys must be copied to the remote server's `/tmp` directory
- **b)** The Private Key must remain strictly confidential on the local client machine; the Public Key is appended to `~/.ssh/authorized_keys` on the remote server
- **c)** The Private Key is sent to the remote server, and the Public Key is deleted immediately
- **d)** Neither key is saved; SSH generates a new password every 10 seconds

> **TIP**
> **Correct Answer: [b]** — The Private Key must remain strictly confidential on the local client machine; the Public Key is appended to `~/.ssh/authorized_keys` on the remote server
> **Technical Explanation:** In asymmetric cryptography, the private key (`id_ed25519`) never leaves the user's local machine and must be protected. The public key (`id_ed25519.pub`) is placed into `~/.ssh/authorized_keys` on the target remote node, allowing the server to challenge the client with a signature verification handshake.

Question 31 [Module 5: Secure Remote Access & Cluster Networking | Medium | Applied Security]

Topic: SSH Strict File Permission Requirements
Question: A student configures passwordless SSH, but the remote cluster repeatedly rejects the key and prompts for a password. Inspecting the remote directory reveals `drwxrwxrwx` (777) on `~/.ssh`. Why does SSH fail?

- **a)** The SSH daemon requires permissions 777 to read the keys
- **b)** OpenSSH enforces strict permission security checks; if `~/.ssh` is group- or world-writable, the SSH daemon considers it compromised and refuses key authentication
- **c)** The student must re-install the Linux operating system
- **d)** OpenSSH does not work on directories with odd octal numbers

> **TIP**
> **Correct Answer: [b]** — OpenSSH enforces strict permission security checks; if `~/.ssh` is group- or world-writable, the SSH daemon considers it compromised and refuses key authentication
> **Technical Explanation:** The OpenSSH daemon (`sshd`) has `StrictModes yes` enabled by default. If `~/.ssh` or `~/.ssh/authorized_keys` is writable by other users or the group (permissions higher than 700 for the folder or 600 for the file), `sshd` refuses to use the keys to prevent unauthorized tampering.

Question 32 [Module 5: Secure Remote Access & Cluster Networking | Easy | Applied Networking]

Topic: Automated SSH Public Key Deployment
Question: Which standard Linux utility automatically creates `~/.ssh` on the remote server with correct permissions (700) and appends the local public key into `~/.ssh/authorized_keys` (600) in a single command?

- **a)** `ssh-copy-id -i ~/.ssh/id_ed25519.pub user@headnode`
- **b)** `scp ~/.ssh/id_ed25519 user@headnode:/etc/passwd`

- **c)** `ssh-keygen -delete user@headnode`
- **d)** `rsync -r ~/.ssh user@headnode:/root`

> **TIP**
> **Correct Answer: [a]** — `ssh-copy-id -i ~/.ssh/id_ed25519.pub user@headnode`
> **Technical Explanation:** `ssh-copy-id` is the standard OpenSSH tool designed specifically to install an authorized public key onto a remote server. It safely creates `~/.ssh` with mode 700 if absent and appends the public key to `authorized_keys` with mode 600, eliminating permission errors.

Question 33 [Module 5: Secure Remote Access & Cluster Networking | Medium | Data Management]

Topic: High-Performance Data Transfer: `rsync` vs `scp`
Question: An HPC dataset transfer of 500 GB was interrupted halfway through due to a temporary network drop. Why is `rsync -avzP` dramatically superior to `scp` for resuming the transfer?

- **a)** `scp` compresses data using quantum algorithms, whereas `rsync` cannot
- **b)** `rsync` utilizes a rolling checksum delta-transfer algorithm with `--partial` to resume exactly where it was interrupted, skipping identical files and transferring only changed byte blocks
- **c)** `rsync` deletes the files on the remote server to save disk space
- **d)** `scp` requires a physical tape drive to resume transfers

> **TIP**
> **Correct Answer: [b]** — `rsync` utilizes a rolling checksum delta-transfer algorithm with `--partial` to resume exactly where it was interrupted, skipping identical files and transferring only changed byte blocks
> **Technical Explanation:** `scp` starts transferring from the very first byte when restarted, wasting hours of transfer time on large datasets. `rsync` with `-P` (`--partial --progress`) retains partially transferred files and inspects block-level checksums, transferring only missing chunks.

Question 34 [Module 5: Secure Remote Access & Cluster Networking | Hard | Cluster Security Architecture]

Topic: SSH Agent Forwarding & Bastion Architecture
Question: In a secure HPC cluster with private compute nodes accessible only through a Bastion / Login node, how does `ssh -A user@login-node` (Agent Forwarding) provide seamless multi-hop authentication without security compromise?

- **a)** It copies the user's private key into `/tmp` on the login node so compute nodes can read it
- **b)** It forwards the local `ssh-agent` authentication socket, allowing the login node to request signature confirmations from the local client machine without exposing the private key to the intermediary node
- **c)** It disables password authentication for all users across the university
- **d)** It converts SSH into an unencrypted telnet connection

> **TIP**
>
> **Correct Answer: [b]** — It forwards the local `ssh-agent` authentication socket, allowing the login node to request signature confirmations from the local client machine without exposing the private key to the intermediary node
> **Technical Explanation:** SSH Agent Forwarding (`-A`) enables multi-hop logins by tunneling authentication challenges back to the client's local memory-resident `ssh-agent` . The private key NEVER leaves the local machine, eliminating the security hazard of leaving private keys on shared jump servers.

Question 35 [Module 5: Secure Remote Access & Cluster Networking | Medium | HPC Networking]

Topic: Network MTU & Jumbo Frames in HPC
Question: Why do high-throughput HPC storage networks and cluster fabrics configure network interfaces with an MTU (Maximum Transmission Unit) of 9000 bytes (Jumbo Frames) instead of the standard 1500 bytes?

- **a)** To allow network cables to stretch 6 times longer
- **b)** To reduce packet fragmentation, lower packet header overhead, and significantly reduce host CPU interrupt processing rates when streaming large datasets
- **c)** Because 9000 bytes is the minimum packet size required by IPv6
- **d)** To prevent unauthorized Wi-Fi connections to the cluster

> **TIP**
>
> **Correct Answer: [b]** — To reduce packet fragmentation, lower packet header overhead, and significantly reduce host CPU interrupt processing rates when streaming large datasets
> **Technical Explanation:** Transferring 1 GB of data with standard 1500-byte MTU requires generating and processing ~700,000 individual network packets, triggering high CPU interrupt rates. With 9000-byte Jumbo Frames, packet count drops to ~115,000 (a 6x reduction), substantially lowering CPU overhead and raising sustained bandwidth.

Question 36 [Module 5: Secure Remote Access & Cluster Networking | Medium | Applied Networking]

Topic: Local Port Forwarding (SSH Tunneling)
Question: A researcher runs a Jupyter Lab notebook on remote compute node port 8888. The compute node has no public IP. What SSH tunneling command must be executed on the researcher's local laptop to view the notebook at `http://localhost:8888` ?

- **a)** `ssh -R 8888:localhost:8888 user@headnode`
- **b)** `ssh -L 8888:localhost:8888 user@headnode`
- **c)** `ssh -D 1080 user@headnode`
- **d)** `ssh -X user@headnode jupyter-lab`

> **TIP**
>
> **Correct Answer: [b]** — `ssh -L 8888:localhost:8888 user@headnode`
> **Technical Explanation:** SSH Local Port Forwarding (`-L <local_port>:<destination_host>:<destination_port>`) listens on the local client machine on port 8888 and forwards all TCP connections through the encrypted SSH tunnel to port 8888 on the target remote node, making remote web UIs accessible locally.

Question 37 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Hard | Scripting Rigor]

Topic: Defensive Bash Scripting: `set -euo pipefail`
Question: Why is it an industry standard to begin all production HPC automation scripts with `set -euo pipefail`?

- **a)** It enables multi-threading and accelerates bash script execution by 400%
- **b)** It ensures the script exits immediately if any command returns a non-zero exit status (`-e`), terminates on uninitialized variable usage (`-u`), and causes pipelines to fail if any sub-command fails (`-o pipefail`)
- **c)** It forces bash to run scripts inside an isolated Docker container
- **d)** It translates the bash script into compiled C++ code automatically

> **TIP**
> **Correct Answer: [b]** — It ensures the script exits immediately if any command returns a non-zero exit status (`-e`), terminates on uninitialized variable usage (`-u`), and causes pipelines to fail if any sub-command fails (`-o pipefail`)
> **Technical Explanation:** By default, bash continues executing subsequent lines even if a command fails, leading to disaster (e.g., `cd /nonexistent; rm -rf *` deletes the wrong files). `set -e` halts on error, `set -u` halts on undefined variables, and `pipefail` catches errors hidden inside pipes (e.g., `broken_cmd | grep foo`).

Question 38 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Hard | Applied Scripting]

Topic: POSIX Signal Trapping & Cluster Cleanup
Question: An HPC simulation creates a 50 GB temporary scratch directory `TMP_DIR=$(mktemp -d /tmp/sim_XXXXXX)` . How can the script guarantee this directory is deleted even if the job is aborted by Ctrl+C or a Slurm preemption signal?

- **a)** By adding `rm -rf $TMP_DIR` as the very last line of the script
- **b)** By registering an exit trap: `trap 'rm -rf "$TMP_DIR"' EXIT INT TERM`
- **c)** By running `chmod 000 $TMP_DIR` before running calculations
- **d)** The Linux kernel automatically deletes all folders named `TMP_DIR` every 10 minutes

> **TIP**
> **Correct Answer: [b]** — By registering an exit trap: `trap 'rm -rf "$TMP_DIR"' EXIT INT TERM`
> **Technical Explanation:** If a script is terminated prematurely by SIGINT (Ctrl+C) or SIGTERM (cluster preemption), standard end-of-file cleanup code is never reached, leaving node scratch disks full. The `trap '...' EXIT INT TERM` command registers an interrupt handler executed unconditionally by the bash runtime upon process exit.

Question 39 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Medium | Scripting Optimization]

Topic: Pure Bash Parameter Expansion
Question: In an automated HPC batch loop, `FILE="/scratch/users/shivam/sim_data_v2.tar.gz"` . How can you extract the filename without its directory path using pure Bash parameter expansion without spawning slow subshells (`basename` / `sed`)?

- **a)** `echo ${FILE%%/*}`
- **b)** `echo ${FILE##*/}`

- **c)** `echo ${FILE%/*}`
- **d)** `echo ${FILE//path/}`

> **TIP**
> **Correct Answer: [b]** — `echo ${FILE##*/}`
> **Technical Explanation:** The `${VAR##pattern}` expansion strips the longest matching pattern from the beginning of the string. Here, `${FILE##*/}` matches everything up to the final slash (`/scratch/users/shivam/`), leaving only `sim_data_v2.tar.gz`. Pure parameter expansion executes in-process in nanoseconds without spawning fork/exec subshells.

Question 40 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Medium | Data Parsing]

Topic: Stream Processing with `awk`
Question: You have an HPC benchmark log file where column 3 contains runtime in seconds. Which `awk` command accurately computes the arithmetic mean (average) runtime across all non-header lines?

- **a)** `awk '{sum += $3} END {print sum/NR}' benchmark.log`
- **b)** `awk 'print avg($3)' benchmark.log`
- **c)** `awk 'SUM($3)/COUNT' benchmark.log`
- **d)** `awk '{print $3 + $3}' benchmark.log`

> **TIP**
> **Correct Answer: [a]** — `awk '{sum += $3} END {print sum/NR}' benchmark.log`
> **Technical Explanation:** In `awk`, `{sum += $3}` increments the variable `sum` by the value of field 3 for every record processed. The `END` block executes after the entire stream has been read, where `sum / NR` divides total sum by `NR` (Number of Records / total lines processed), printing the exact arithmetic mean.

Question 41 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Medium | Shell Internals]

Topic: I/O Redirection & File Descriptors
Question: What is the critical behavioral difference between `cmd > log.txt 2>&1` and `cmd 2>&1 > log.txt` in Linux bash?

- **a)** Both commands perform the exact same redirection
- **b)** `cmd > log.txt 2>&1` redirects both stdout and stderr to `log.txt`; `cmd 2>&1 > log.txt` redirects stderr to the terminal screen and only stdout to `log.txt`
- **c)** `cmd 2>&1 > log.txt` creates two separate files
- **d)** `cmd > log.txt 2>&1` is invalid syntax in modern Linux

> **TIP**
> **Correct Answer: [b]** — `cmd > log.txt 2>&1` redirects both stdout and stderr to `log.txt`; `cmd 2>&1 > log.txt` redirects stderr to the terminal screen and only stdout to `log.txt`
> **Technical Explanation:** Redirections are evaluated left to right. In `2>&1 > log.txt`: fd 2 (stderr) is duplicated to fd 1's current destination (the terminal screen), then fd 1 (stdout) is redirected to `log.txt`. In `> log.txt 2>&1`: fd 1 is redirected to `log.txt`, then fd 2 is duplicated to point to `log.txt` as well, successfully capturing all output.

Question 42 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Hard | Applied Scripting]

Topic: Bash Associative Arrays for Node Tracking
Question: How do you declare and populate an associative array (hash map / key-value dictionary) in Bash 4+ to track GPU counts per cluster node?

- **a)** `set -A GPU_MAP; GPU_MAP[node01]=4`
- **b)** `declare -A GPU_MAP; GPU_MAP["node01"]=4; GPU_MAP["node02"]=8`
- **c)** `GPU_MAP = {node01: 4, node02: 8}`
- **d)** `array GPU_MAP = (node01=4, node02=8)`

> **TIP**
> **Correct Answer: [b]** — `declare -A GPU_MAP; GPU_MAP["node01"]=4; GPU_MAP["node02"]=8`
> **Technical Explanation:** Bash associative arrays require explicit declaration using `declare -A ARRAY_NAME`. Values are assigned using string keys: `GPU_MAP["node01"]=4`. Attempting to use string keys without `-A` treats strings as arithmetic expressions evaluating to index 0, corrupting data.

Question 43 [Module 6: Advanced Linux Bash Shell Scripting for HPC Automation | Hard | Automation Architecture]

Topic: Dynamic Batch Job Generation (Here-Documents)
Question: An engineer writes a script to dynamically generate a Slurm batch script. Why is using `cat << 'EOF' > job.sh` (quoted delimiter) fundamentally different from `cat << EOF > job.sh` (unquoted delimiter)?

- **a)** The quoted delimiter executes the script in python mode
- **b)** Quoting `'EOF'` prevents the current shell from expanding variables (e.g., `$SLURM_JOB_ID`), preserving variable literals to be evaluated later on the compute node
- **c)** Unquoted `EOF` permanently encrypts the generated file
- **d)** Quoting `'EOF'` limits the file size to 10 lines

> **TIP**
> **Correct Answer: [b]** — Quoting `'EOF'` prevents the current shell from expanding variables (e.g., `$SLURM_JOB_ID`), preserving variable literals to be evaluated later on the compute node
> **Technical Explanation:** When the delimiter is unquoted (`<< EOF`), the launching bash shell immediately evaluates `$VAR` expressions before writing. If `$SLURM_JOB_ID` is not yet set in the launching environment, it expands to an empty string. Quoting the delimiter (`<< 'EOF'`) disables expansion, writing literal `$SLURM_JOB_ID` to the script.

Question 44 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Easy | Programming Models]

Topic: oneAPI Cross-Architecture Vision
Question: What primary industry challenge does the Intel oneAPI open programming specification directly solve for high-performance computing?

- **a)** It forces all computers worldwide to run the same operating system
- **b)** It eliminates proprietary vendor-locked programming models (e.g., CUDA) by delivering a unified, standards-based C++/SYCL programming model across heterogeneous CPUs, GPUs, FPGAs, and AI accelerators
- **c)** It replaces standard C++ with a new proprietary proprietary binary format
- **d)** It allows computer hardware to operate without compilers

> **TIP**
> **Correct Answer: [b]** — It eliminates proprietary vendor-locked programming models (e.g., CUDA) by delivering a unified, standards-based C++/SYCL programming model across heterogeneous CPUs, GPUs, FPGAs, and AI accelerators
> **Technical Explanation:** Historically, developers had to maintain completely separate codebases for CPUs (OpenMP), GPUs (CUDA), and FPGAs (Verilog/OpenCL). Intel oneAPI delivers an open, cross-vendor programming model based on Khronos SYCL, enabling a single codebase to target heterogeneous compute hardware.

Question 45 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Medium | oneAPI Architecture]

Topic: oneAPI Base Toolkit vs HPC Toolkit
Question: What is the key functional distinction between the Intel oneAPI Base Toolkit and the Intel oneAPI HPC Toolkit?

- **a)** The Base Toolkit is free, while the HPC Toolkit is only for video games
- **b)** The Base Toolkit contains core developer tools (DPC++/C++ compiler, oneMKL, oneDNN, VTune Profiler); the HPC Toolkit adds distributed cluster tools (Intel Fortran, Intel MPI Library, Cluster Checker)
- **c)** The Base Toolkit only works on Windows; the HPC Toolkit only works on macOS
- **d)** The Base Toolkit compiles code; the HPC Toolkit only monitors hardware temperature

> **TIP**
> **Correct Answer: [b]** — The Base Toolkit contains core developer tools (DPC++/C++ compiler, oneMKL, oneDNN, VTune Profiler); the HPC Toolkit adds distributed cluster tools (Intel Fortran, Intel MPI Library, Cluster Checker)
> **Technical Explanation:** Intel packages oneAPI modularly. The Base Toolkit provides the foundational heterogeneous tools (Intel oneAPI DPC++/C++ Compiler, oneMKL math library, oneDNN, VTune, Advisor). The HPC Toolkit layers on enterprise distributed computing essentials: Intel MPI Library, Intel Fortran Compiler, and Intel Inspector.

Question 46 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Easy | Toolchain Configuration]

Topic: Environment Initialization (`setvars.sh`)
Question: Why must an engineer execute `source /opt/intel/oneapi/setvars.sh` in their shell before invoking oneAPI compilers (`icpx` , `ifx`) or tools?

- **a)** To download the latest compiler updates from the internet

- **b)** To initialize mandatory environment variables (`ONEAPI_ROOT` , `PATH` , `CPATH` , `LD_LIBRARY_PATH` , `CMAKE_PREFIX_PATH`) pointing to active toolkit installations
- **c)** To reboot the Linux kernel into HPC mode
- **d)** To delete old temporary files in `/tmp`

> **TIP**
> **Correct Answer: [b]** — To initialize mandatory environment variables (`ONEAPI_ROOT` , `PATH` , `CPATH` , `LD_LIBRARY_PATH` , `CMAKE_PREFIX_PATH`) pointing to active toolkit installations
> **Technical Explanation:** oneAPI installs compilers and libraries into modular directories under `/opt/intel/oneapi/` . Sourcing `setvars.sh` runs environment setup scripts that export essential paths to `PATH` (for binaries like `icpx`), `CPATH` (for header files like `sycl/sycl.hpp`), and `LD_LIBRARY_PATH` (for shared `.so` runtimes).

Question 47 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Medium | SYCL Architecture]

Topic: SYCL Single-Source Heterogeneous Model
Question: In SYCL and DPC++ programming, what does the 'Single-Source' programming paradigm mean?

- **a)** All source code across an entire university must be written in a single file
- **b)** Both host execution code (orchestration, I/O) and device execution kernels (vector arithmetic) are written together within the same C++ file and compiled by a unified compiler toolchain
- **c)** The code can only read data from one single input file
- **d)** Only one developer can edit the code repository at a time

> **TIP**
> **Correct Answer: [b]** — Both host execution code (orchestration, I/O) and device execution kernels (vector arithmetic) are written together within the same C++ file and compiled by a unified compiler toolchain
> **Technical Explanation:** Unlike OpenCL (which required runtime string compilation of separate kernel files) or early CUDA, SYCL allows standard modern C++ lambda expressions and functions to define accelerator device kernels directly alongside host management code in a single `.cpp` file, compiled seamlessly by `icpx -fsycl` .

Question 48 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Medium | SYCL Runtime]

Topic: SYCL Queue & Device Selectors
Question: Examine the following C++ SYCL code snippet: `sycl::queue q(sycl::gpu_selector_v);` . What happens at runtime if this program is executed on an HPC server that has no GPU installed?

- **a)** The computer automatically orders a GPU from the internet
- **b)** The SYCL runtime throws a `sycl::exception` indicating no matching device was discovered
- **c)** The kernel executes silently on the CPU without notifying the user
- **d)** The operating system crashes and reboots

> **TIP**
> **Correct Answer: [b]** — The SYCL runtime throws a `sycl::exception` indicating no matching device was discovered
> **Technical Explanation:** The `gpu_selector_v` explicitly mandates a physical GPU target. If no supported GPU device is discovered by the OpenCL/Level Zero runtime drivers, the constructor throws a synchronous `sycl::exception`. To allow automatic fallback to CPU when no GPU is present, `default_selector_v` should be used instead.

Question 49 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Hard | Memory Management in SYCL]

Topic: SYCL Unified Shared Memory (USM) Allocations
Question: What is the key technical difference between `sycl::malloc_shared()` and `sycl::malloc_device()` when managing memory in a heterogeneous SYCL application?

- **a)** `malloc_shared` allocates memory on the hard drive; `malloc_device` allocates memory in CPU registers
- **b)** `malloc_shared` provides memory accessible by both host CPU and accelerator device with automatic hardware/driver page migration; `malloc_device` allocates high-bandwidth memory residing strictly on the accelerator, inaccessible by host CPU pointers
- **c)** `malloc_device` is only for Intel CPUs; `malloc_shared` is only for NVIDIA GPUs
- **d)** There is no difference; they are synonymous macros in SYCL 2020

> **TIP**
> **Correct Answer: [b]** — `malloc_shared` provides memory accessible by both host CPU and accelerator device with automatic hardware/driver page migration; `malloc_device` allocates high-bandwidth memory residing strictly on the accelerator, inaccessible by host CPU pointers
> **Technical Explanation:** In Unified Shared Memory (USM): `malloc_shared` allocates memory accessible by both host and device pointers, with the runtime automatically migrating virtual memory pages on demand across the PCIe bus. `malloc_device` allocates memory strictly on the device (e.g., GPU HBM2e) for maximum bandwidth; dereferencing it on the host CPU causes a segmentation fault.

Question 50 [Module 7: Intel oneAPI Ecosystem & Heterogeneous Programming Foundations | Medium | Compilation & Toolchains]

Topic: Compiling SYCL with Intel DPC++/C++ Compiler (`icpx`)
Question: Which exact compiler invocation compiles a SYCL C++ program `dpcpp_vector_add.cpp` into an optimized native binary using Intel oneAPI on Linux?

- **a)** `gcc -O3 dpcpp_vector_add.cpp -o vector_add`
- **b)** `icpx -fsycl -O3 dpcpp_vector_add.cpp -o vector_add`
- **c)** `python -m compile dpcpp_vector_add.cpp`
- **d)** `make -sycl dpcpp_vector_add.cpp`

> **TIP**
>
> **Correct Answer: [b]** — `icpx -fsycl -O3 dpcpp_vector_add.cpp -o vector_add`
>
> **Technical Explanation:** The Intel oneAPI DPC++/C++ compiler executable is `icpx`. The `-fsycl` flag is mandatory because it instructs the Clang/LLVM frontend to parse SYCL 2020 constructs, generate host code, and invoke the device code-generation backend for target architectures. `-O3` enables aggressive loop vectorization.
